

容器资源管理

容器资源管理

- 容器资源管理
 - 资源共享
 - 限制运行时的 CPU 资源
 - 限制运行时的内存
 - 限制运行时的 IO 资源
 - 限制容器 rootfs 存储空间
 - 限制容器内文件句柄数
 - 限制容器内可以创建的进程/线程数
 - 配置容器内的 ulimit 值

资源共享

描述

容器间或者容器与 host 之间可以共享 namespace 信息，包括 pid, net, ipc, uts。

用法

isula create/run 时使用 namespace 相关的参数共享资源，具体参数见下方参数列表。

参数

create/run 时可以指定下列参数。

参数项

参数说明

取值范围

是否必选

--pid

指定要共享的 pid namespace

[none, host, container:], none 表示不共享, host 表示与 host 共用 namespace, container:表示与容器 containerID 共享同一个 namespace

否

--net

指定要共享的 net namespace

[none, host, container:], none 表示不共享, host 表示与 host 共用 namespace, container:表示与容器 containerID 共享同一个 namespace

否

--ipc

指定要共享的 ipc namespace

[none, host, container:], none 表示不共享, host 表示与 host 共用 namespace, container:表示与容器 containerID 共享同一个 namespace

否

--uts

指定要共享的 uts namespace

[none, host, container:], none 表示不共享, host 表示与 host 共用 namespace, container:表示与容器 containerID 共享同一个 namespace

否

示例

如果两个容器需要共享同一个 pid namespace, 在运行容器时, 直接加上--pid container: 即可, 如:

```
isula run -tid --name test_pid busybox sh
```

```
isula run -tid --name test --pid container:test_pid busybox sh
```

限制运行时的 CPU 资源

描述

可以通过参数限制容器的各项 cpu 资源值。

用法

isula create/run 时使用 cpu 相关的参数限制容器的各项 cpu 资源值，具体参数及取值见下方参数列表。

参数

create/run 时可以指定下列参数。

参数项

参数说明

取值范围

是否必选

--cpu-period

限制容器中 cpu cfs（完全公平调度）周期

64 位整数(int64)

否

--cpu-quota

限制容器中 cpu cfs(完全公平调度) 的配额

64 位整数(int64)

否

--cpu-shares

限制容器中 cpu 相对权重

64 位整数(int64)

否

--cpuset-cpus

限制容器中使用 cpu 节点

字符串。值为要设置的 cpu 编号，有效范围为主机上的 cpu 数量，例如可以设置 0-3 或者 0,1.

否

`--cpuset-mems`

限制容器中 cpuset 使用的 mem 节点

字符串。值为要设置的 cpu 编号，有效范围为主机上的 cpu 数量，例如可以设置 0-3 或者 0,1.

否

示例

如果需要限制容器只是用特定的 cpu，在运行容器时，直接加上`--cpuset-cpus number`即可，如：

```
isula run -tid --cpuset-cpus 0,2-3 busybox sh
```

说明： 是否设置成功，请参见“查询单个容器信息”章节。

限制运行时的内存

描述

可以通过参数限制容器的各项内存值上限。

用法

isula create/run 时使用内存相关的参数限制容器的各项内存使用上限，具体参数及取值见下方参数列表。

参数

create/run 时可以指定下列参数。

参数项

参数说明

取值范围

是否必选

`--memory`

限制容器中内存使用上限

64 位整数(int64)。值为非负数，0 表示不设置（不限制）；单位可以为空 (byte)，KB，MB，GB，TB，PB.

否

`--memory-reservation`

限制容器中内存的软上限

64 位整数(int64)。值为非负数，0 表示不设置（不限制）；单位可以为空 (byte)，KB，MB，GB，TB，PB.

否

`--memory-swap`

限制容器中交换内存的上限

64 位整数(int64)。值为-1 或非负数，-1 表示不限制，0 表示不设置（不限制）；单位可以为空(byte)，KB，MB，GB，TB，PB.

否

`--kernel-memory`

限制容器中内核内存的上限

64 位整数(int64)。值为非负数，0 表示不设置（不限制）；单位可以为空 (byte)，KB，MB，GB，TB，PB.

否

示例

如果需要限制容器内内存的上限，在运行容器时，直接加上`--memory []`即可，如：

```
isula run -tid --memory 1G busybox sh
```

限制运行时的 IO 资源

描述

可以通过参数限制容器中设备读写速度。

用法

isula create/run 时使用--device-read-bps/--device-write-bps :[]来限制容器中设备的读写速度。

参数

create/run 时指定--device-read/write-bps 参数。

参数项

参数说明

取值范围

是否必选

--device-read-bps/--device-write-bps

限制容器中设备的读速度/写速度

64 位整数(int64)。值为正整数，可以为 0，0 表示不设置（不限制）；单位可以为空 (byte)，KB，MB，GB，TB，PB.

否

示例

如果需要限制容器内设备的读写速度，在运行容器时，直接加上--device-write-bps/--device-read-bps :[]即可，例如，限制容器 busybox 内设备/dev/sda 的读速度为 1MB 每秒，则命令如下：

```
isula run -tid --device-write /dev/sda:1mb busybox sh
```

限制写速度的命令如下：

```
isula run -tid read-bps /dev/sda:1mb busybox sh
```

限制容器 rootfs 存储空间

描述

在 ext4 上使用 overlay2 时，可以设置单个容器的文件系统限额，比如设置 A 容器的限额为 5G，B 容器为 10G。

该特性通过 ext4 文件系统的 project quota 功能来实现，在内核支持的前提下，通过系统调用 SYS_IOCTL 设置某个目录的 project ID，再通过系统调用 SYS_QUOTACTL 设置相应的 project ID 的 hard limit 和 soft limit 值达到限额的目的。

用法

1. 环境准备

文件系统支持 Project ID 和 Project Quota 属性，4.19 版本内核已经支持，外围包 e2fsprogs 版本不低于 1.43.4-2。

2. 在容器挂载 overlayfs 之前，需要对不同容器的 upper 目录和 work 目录设置不同的 project id，同时设置继承选项，在容器挂载 overlayfs 之后不允许再修改 project id 和继承属性。

3. 配额的设置需要在容器外以特权用户进行。

4. daemon 中增加如下配置

```
-s overlay2 --storage-opt overlay2.override_kernel_check=true
```

5. daemon 支持以下选项，用于为容器设置默认的限制，

--storage-opt overlay2.basesize=128M 指定默认限制的大小，若 isula run 时也指定了 --storage-opt size 选项，则以 run 时指定来生效，若 daemon 跟 isula run 时都不指定大小，则表示不限制。

6. 需要开启文件系统 Project ID 和 Project Quota 属性。

– 新格式化文件系统并 mount

```
# mkfs.ext4 -O quota,project /dev/sdb  
# mount -o prjquota /dev/sdb /var/lib/isulad
```

参数

create/run 时指定 --storage-opt 参数。

参数项

参数说明

取值范围

是否必选

```
--storage-opt size=${rootfsSize}
```

限制容器 rootfs 存储空间。

rootfsSize 解析出的大小为 int64 范围内以字节表示的正数，默认单位为 B，也可指定为 ([kKmMgGtTpP])?[il]?[bB]?\$

否

示例

在 isula run/create 命令行上通过已有参数 “--storage-opt size=” 来设置限额。其中 value 是一个正数，单位可以是[kKmMgGtTpP])?[il]?[bB]?，在不带单位的时候默认单位是字节。

```
# isula run -ti --storage-opt size=10M busybox
/ # df -h
Filesystem      Size  Used Available Use% Mounted on
overlay         10.0M  48.0K   10.0M   0% /
none            64.0M    0   64.0M   0% /dev
none            10.0M    0   10.0M   0% /sys/fs/cgroup
tmpfs           64.0M    0   64.0M   0% /dev
shm             64.0M    0   64.0M   0% /dev/shm
/dev/mapper/vg--data-ext41
          9.8G  51.5M   9.2G   1% /etc/hostname
/dev/mapper/vg--data-ext41
          9.8G  51.5M   9.2G   1% /etc/resolv.conf
/dev/mapper/vg--data-ext41
          9.8G  51.5M   9.2G   1% /etc/hosts
tmpfs           3.9G    0   3.9G   0% /proc/acpi
tmpfs           64.0M    0   64.0M   0% /proc/kcore
tmpfs           64.0M    0   64.0M   0% /proc/keys
tmpfs           64.0M    0   64.0M   0% /proc/timer_list
tmpfs           64.0M    0   64.0M   0% /proc/sched_debug
tmpfs           3.9G    0   3.9G   0% /proc/scsi
tmpfs           64.0M    0   64.0M   0% /proc/fdthreshold
tmpfs           64.0M    0   64.0M   0% /proc/fdenable
tmpfs           3.9G    0   3.9G   0% /sys/firmware
/ #
/ # dd if=/dev/zero of=/home/img bs=1M count=12 && sync
dm-4: write failed, project block limit reached.
10+0 records in
```



```
9+0 records out
10432512 bytes (9.9MB) copied, 0.011782 seconds, 844.4MB/s
/ # df -h | grep overlay
overlay          10.0M   10.0M    0 100% /
/ #
```

约束

1. 限额只针对 rw 层。

overlay2 的限额是针对容器的 rw 层的，镜像的大小不计算在内。

- ## 2. 内核支持并使能。

内核必须支持 ext4 的 project quota 功能，并在 mkfs 的时候要加上 -O quota,project，挂载的时候要加上 -o prjquota。任何一个不满足，在使用 --storage-opt size= 时都将报错。

```
# isula run -it --storage-opt size=10Mb busybox df -h
Error response from daemon: Failed to prepare rootfs with error: time="2019-04-09T05:13:52-04:00" level=fatal msg="error creating read-write layer with ID
"a4c0e55e82c55e4ee4b0f4ee07f80cc2261cf31b2c2dfd628fa1fb00db97270f": --
storage-opt is supported only for overlay over
xfs or ext4 with 'pquota' mount option"
```

- ### 3. 限制额度的说明。

- [illegible]

4. 上一次启动 isulad 时，isulad 的 root 所在分区挂载时加了 -o priquota 选项，这次启动时不加，那么上一次启动中创建的带 quota 的容器的设置值不生效。
 5. daemon 端配额 --storage-opt overlay2.basesize，其取值范围与 --storage-opt size 相同。
4. 指定 storage-opt 为 4k 时，轻量级容器启动与 docker 有差异

使用选项 storage-opt size=4k 和镜像

rnd-dockerhub.huawei.com/official/ubuntu-arm64:latest 运行容器。

docker 启动失败。

```
# docker run -itd --storage-opt size=4k rnd-dockerhub.huawei.com/official/ubuntu-arm64:latest
docker: Error response from daemon: symlink /proc/mounts
/var/lib/docker/overlay2/e6e12701db1a488636c881b44109a807e187b8db51a50015
db34a131294fcf70-init/merged/etc/mtab: disk quota exceeded.
See 'docker run --help'.
```

轻量级容器不报错，正常启动

```
# isula run -itd --storage-opt size=4k rnd-dockerhub.huawei.com/official/ubuntu-arm64:latest
636480b1fc2cf8ac895f46e77d86439fe2b359a1ff78486ae81c18d089bbd728
# isula ps
STATUS PID IMAGE COMMAND EXIT_CODE
RESTART_COUNT STARTAT FINISHAT RUNTIME ID NAMES
running 17609 rnd-dockerhub.huawei.com/official/ubuntu-arm64:latest /bin/bash 0
0 2 seconds ago - lcr 636480b1fc2c
636480b1fc2cf8ac895f46e77d86439fe2b359a1ff78486ae81c18d089bbd728
```

在启动容器的过程中，如果需要在容器的 rootfs 路径下创建文件，若镜像本身占用的大小超过 4k，且此时的 quota 设置为 4k，则创建文件必定失败。

docker 在启动容器的过程中，会比 isulad 创建更多的挂载点，用于挂载 host 上的某些路径到容器中，如 /proc/mounts, /dev/shm 等，如果镜像内本身不存在这些文件，则会创建，根据上述原因该操作会导致文件创建失败，因而容器启动失败。

轻量级容器在启动容器过程中，使用默认配置时，挂载点较少，如 /proc，或 /sys 等路径不存在时，才会创建。用例中的镜像 rnd-dockerhub.huawei.com/

official/ubuntu-arm64:latest 本身含有 /proc， /sys 等，因此整个启动容器的过程中，都不会有新文件或路径生成，故轻量级容器启动过程不会报错。为验证这一过程，当把镜像替换为 rnd-dockerhub.huawei.com/official/busybox-aarch64:latest 时，由于该镜像内无 /proc 存在，轻量级容器启动一样会报错。

```
# isula run -itd --storage-opt size=4k rnd-dockerhub.huawei.com/official/busybox-aarch64:latest
8e893ab483310350b8caa3b29eca7cd3c94eae55b48bfc82b350b30b17a0aaf4
Error response from daemon: Start container error: runtime error:
8e893ab483310350b8caa3b29eca7cd3c94eae55b48bfc82b350b30b17a0aaf4:tools/
lxc_start.c:main:404 starting container process caused "Failed to setup lxc,
please check the config file."
```

5. 其他说明。

使用限额功能的 isulad 切换数据盘时，需要保证被切换的数据盘使用 `prjquota` 选项挂载，且 /var/lib/isulad/storage/overlay2 目录的挂载方式与 /var/lib/isulad 相同。

说明： 切换数据盘时需要保证 /var/lib/isulad/storage/overlay2 的挂载点被卸载。

限制容器内文件句柄数

描述

可以通过参数限制容器中可以打开的文件句柄数。

用法

isula create/run 时使用 --files-limit 来限制容器中可以打开的文件句柄数。

参数

create/run 时指定 --files-limit 参数。

参数项

参数说明

取值范围

是否必选

--files-limit

限制容器中可以打开的文件句柄数。

64 位整数(int64)。可以为 0、负，但不能超过 2 的 63 次方减 1，0、负表示不做限制（max）。由于创建容器的过程中会临时打开一些句柄，所以此值不能设置的太小，不然容器可能不受 files limit 的限制（如果设置的数小于当前已经打开的句柄数，会导致 cgroup 文件写不进去），建议大于 30。

否

示例

在运行容器时，直接加上--files-limit n 即可，如：

```
isula run -ti --files-limit 1024 busybox bash
```

约束

1. 使用--files-limit 参数传入一个很小的值，如 1，可能导致容器启动失败。

```
# isula run -itd --files-limit 1 rnd-dockerhub.huawei.com/official/busybox-aarch64
004858d9f9ef429b624f3d20f8ba12acfb8a15bb121c4036de4e5745932eff4
Error response from daemon: Start container error: Container is not
running:004858d9f9ef429b624f3d20f8ba12acfb8a15bb121c4036de4e5745932eff4
```

而 docker 会启动成功，其 files.limit cgroup 值为 max。

```
# docker run -itd --files-limit 1 rnd-dockerhub.huawei.com/official/busybox-aarch64
ef9694bf4d8e803a1c7de5c17f5d829db409e41a530a245edc2e5367708dbbab
# docker exec -it ef96 cat /sys/fs/cgroup/files/files.limit
max
```

根因是 lxc 和 runc 启动过程的原理不一样，lxc 创建 cgroup 子组后先设置 files.limit 值，再将容器进程的 PID 写入该子组的 cgroup.procs 文件，此时该进程已经打开超过 1 个句柄，因而写入报错导致启动失败。runc 创建 cgroup 子组后先将容器进程的 PID 写入该子组的 cgroup.procs 文件，再设置 files.limit 值，此时由于该子组内的进程已经打开超过 1 个句柄，因而写入 files.limit 不会生效，内核也不会报错，容器启动成功。

限制容器内可以创建的进程-线程数

描述

可以通过参数限制容器中能够创建的进程/线程数。

用法

在容器 create/run 时，使用参数--pids-limit 来限制容器中可以创建的进程/线程数。

参数

create/run 时指定--pids-limit 参数。

参数项

参数说明

取值范围

是否必选

--pids-limit

限制容器中可以打开的文件句柄数。

64 位整数(int64)。可以为 0、负，但不能超过 2 的 63 次方减 1，0、负表示不做限制（max）。

否

示例

在运行容器时，直接加上--pids-limit n 即可，如：

```
isula run -ti --pids-limit 1024 busybox bash
```

约束

由于创建容器的过程中会临时创建一些进程，所以此值不能设置的太小，不然容器可能起不来，建议大于 10。

配置容器内的 ulimit 值

描述

可以通过参数控制执行程序的资源。

用法

在容器 create/run 时配置--ulimit 参数，或通过 daemon 端配置，控制容器中执行程序的资源。

参数

通过两种方法配置 ulimit

1. isula create/run 时使用--ulimit =[:]来控制 shell 执行程序的资源。

参数项

参数说明

取值范围

是否必选

--ulimit

限制 shell 执行程序的资源

soft/hard 是 64 位整数(int64)。soft 取值 <= hard 取值，如果仅仅指定了 soft 的取值，则 hard=soft。对于某些类型的资源并不支持负数，详见下表

否

2. 通过 daemon 端参数或配置文件

详见"(命令行参数说明"与"部署方式"的--default-ulimits 相关选项。

--ulimit 可以对以下类型的资源进行限制。

类型

说明

取值范围

core

limits the core file size (KB)

64 位整数(INT64)，无单位。可以为 0、负、其中-1 表示 UNLIMITED，即不做限制，其余的负数会被强制转换为一个大的正整数。

cpu

max CPU time (MIN)

data

max data size (KB)

fsize

maximum filesize (KB)

locks

max number of file locks the user can hold

memlock

max locked-in-memory address space (KB)

msgqueue

max memory used by POSIX message queues (bytes)

nice

nice priority

nproc

max number of processes

rss

max resident set size (KB)

rtprio

max realtime priority

rttime

realtime timeout

sigpending

max number of pending signals

stack

max stack size (KB)

nofile

max number of open file descriptors

64 位整数(int64)，无单位。不可以为负，负数被强转为大数，设置时会出现
Operation not permitted

示例

在容器的创建或者运行时，加上--ulimit =[:]即可，如：

```
isula create/run -tid --ulimit nofile=1024:2048 busybox sh
```

约束

不能在 daemon.json 和 /etc/sysconfig/iSulad 文件（或 isulad 命令行）中同时配置 ulimit 限制，否则 isulad 启动会报错。